Unit III Searching and Sorting

Prof. Pranjal Pandit

Searching and Sorting

Searching:

- sequential/linear search,
- sentinel search,
- binary search,
- Fibonacci search,
- indexed sequential search.

Sorting:

- internal vs external sorting;
- sort order, stability, efficiency and number of passes;
- comparison-based methods (Bubble, Insertion, Selection, Quick, Shell);
- non-comparison-based methods (Radix, Counting, Bucket);
- comparison of all methods and their complexities.

Non-Comparison based sorting

- Counting
- Bucket
- Radix

Counting Sort

- Counting Sort is a **non-comparison-based** sorting algorithm. It is particularly efficient when the **range of input values is small** compared to the number of elements to be sorted.
- The basic idea behind Counting Sort is **to count the frequency** of each **distinct element** in the input array and use that information to place the elements in their correct sorted positions.
- It works well when the range of input elements is small and comparable to the size of the array.

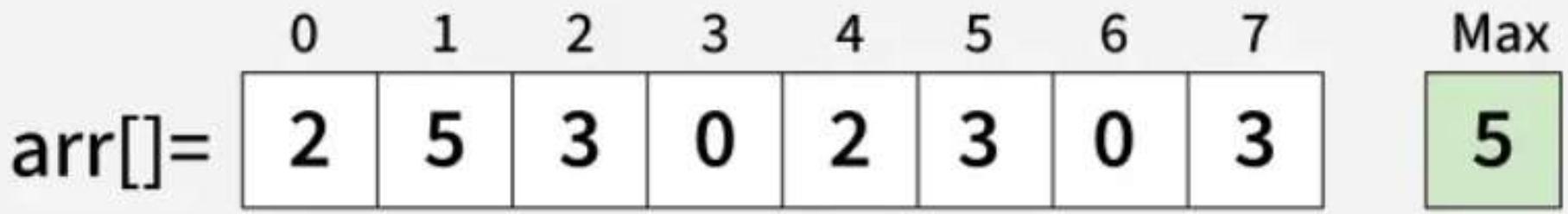
For example, for input [1, 4, 0, 2, 1, 1, 2, 4], the size of array is 8 and range of elements is from 0 to 4

- If range of input array is of order more than n Log n where n is size of the array, then we can better sort the array using a standard comparison based sorting algorithm like Merge Sort.

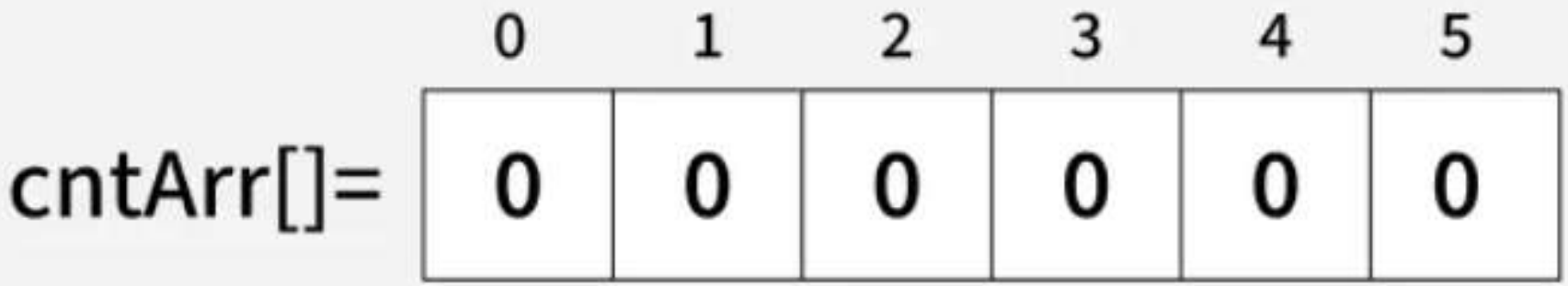
Counting Sort Algorithm

- Declare a count array **cntArr[]** of size **max(arr[])+1** and initialize it with **0**.
- Traverse input array **arr[]** and map each element of **arr[]** as an index of **cntArr[]** array, i.e., execute **cntArr[arr[i]]++** for **0 <= i < N**.
- Calculate the prefix sum at every index of **cntArr[]**.
- Create an array **ans[]** of size **N**.
- Traverse array **arr[]** from end and update **ans[cntArr[arr[i]] - 1] = arr[i]**. Also, update cntArr**[arr[i]] = cntArr[arr[i]]- -** .

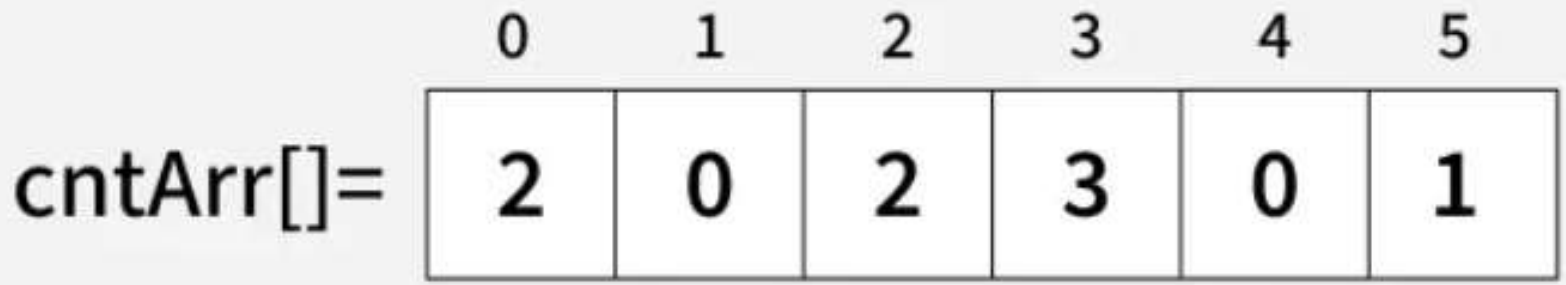
Find out the maximum element from the given array.



Initialize a cntArr[] of length max+1 with all elements as 0. This array will be used for storing the occurrences of the elements of the input array.



In the cntArr[], store the count of each unique element of the input array at their respective indices.

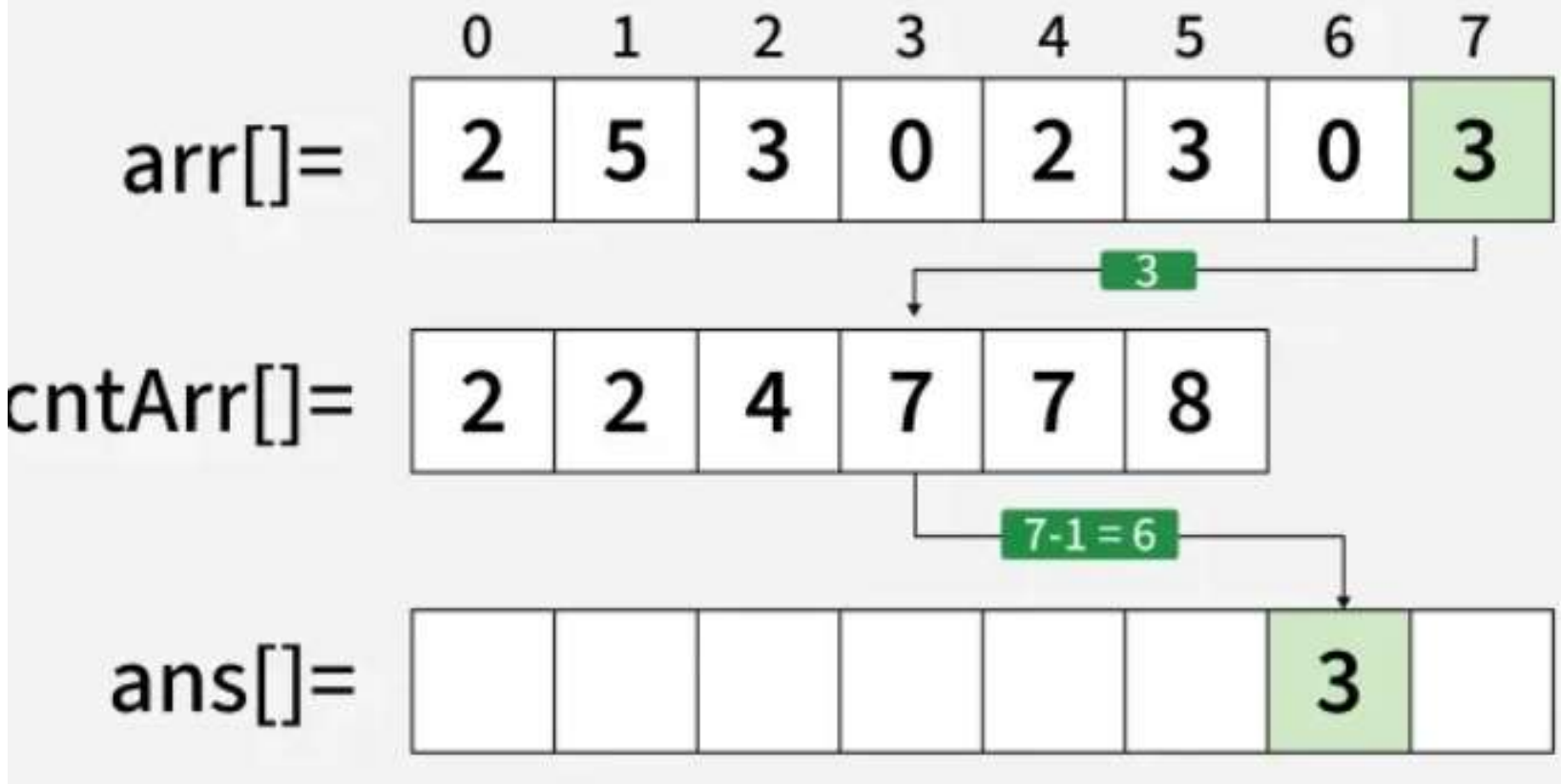


Store the cumulative sum or prefix sum of the elements of the cntArr[] by doing cntArr[i] = cntArr[i - 1] + cntArr[i].

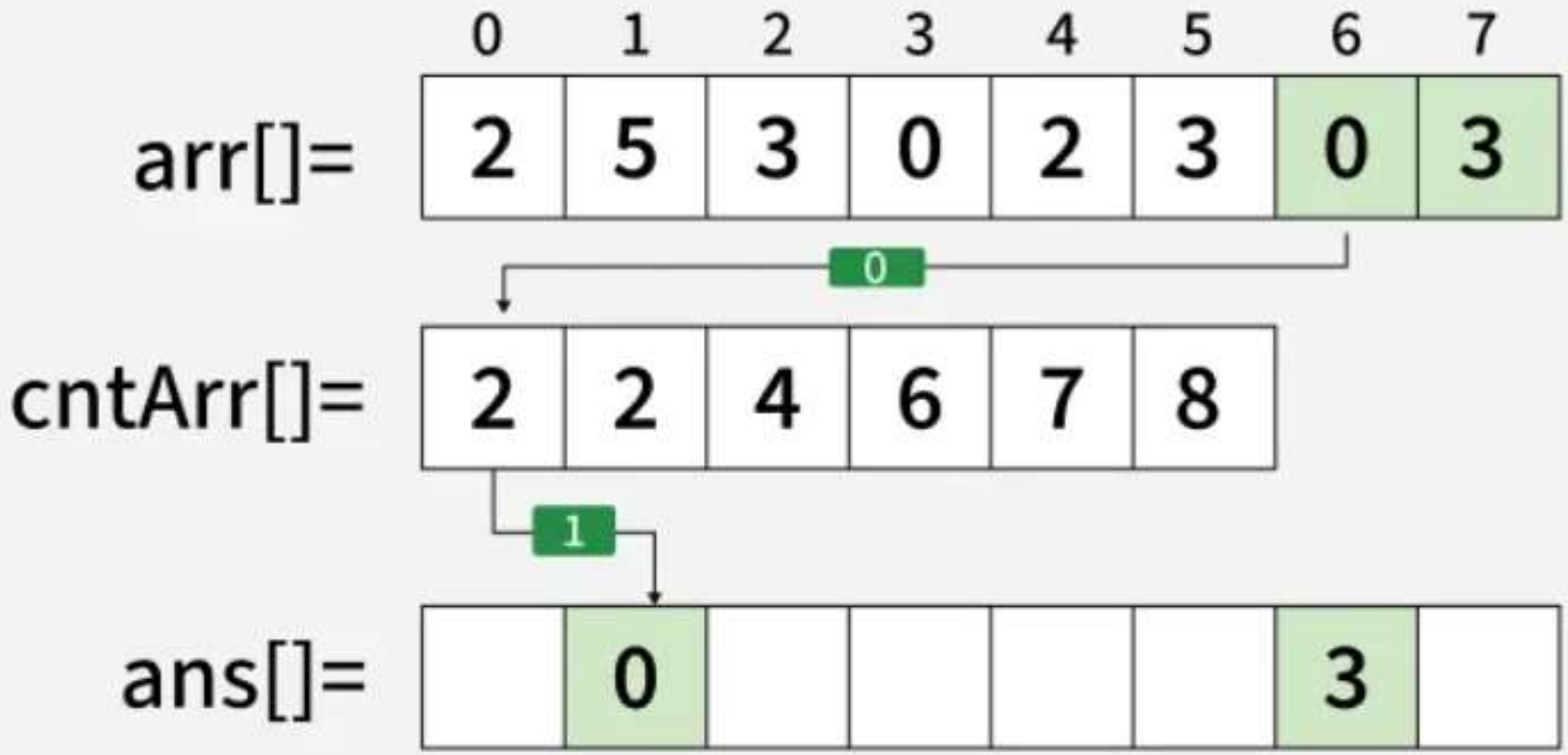
cntArr[]=

0	1	2	3	4	5
2	2	4	7	7	8

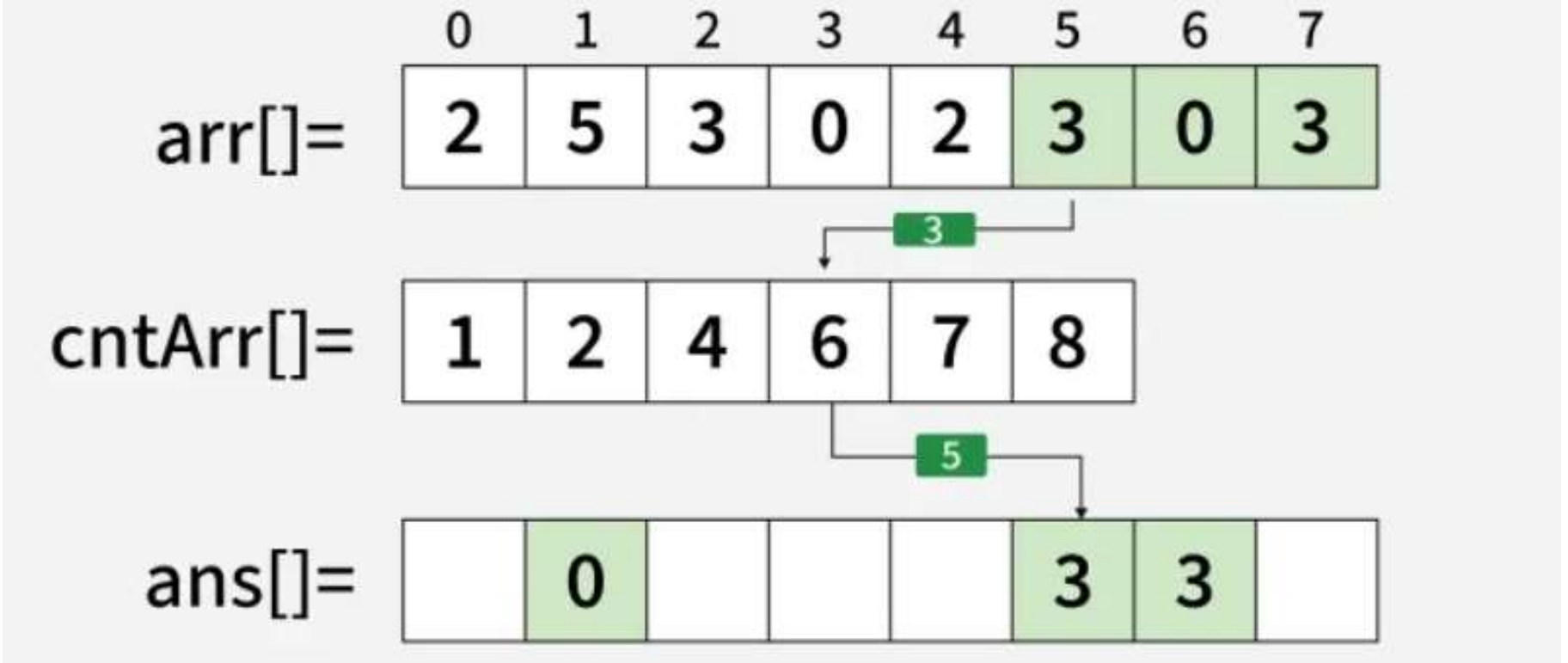
Update ans[cntArr[arr[i]] - 1] = arr[i] and decrement cntArr[arr[i]]. Traverse the input array in reverse to maintain the order of equal elements, ensuring the sort remains stable.



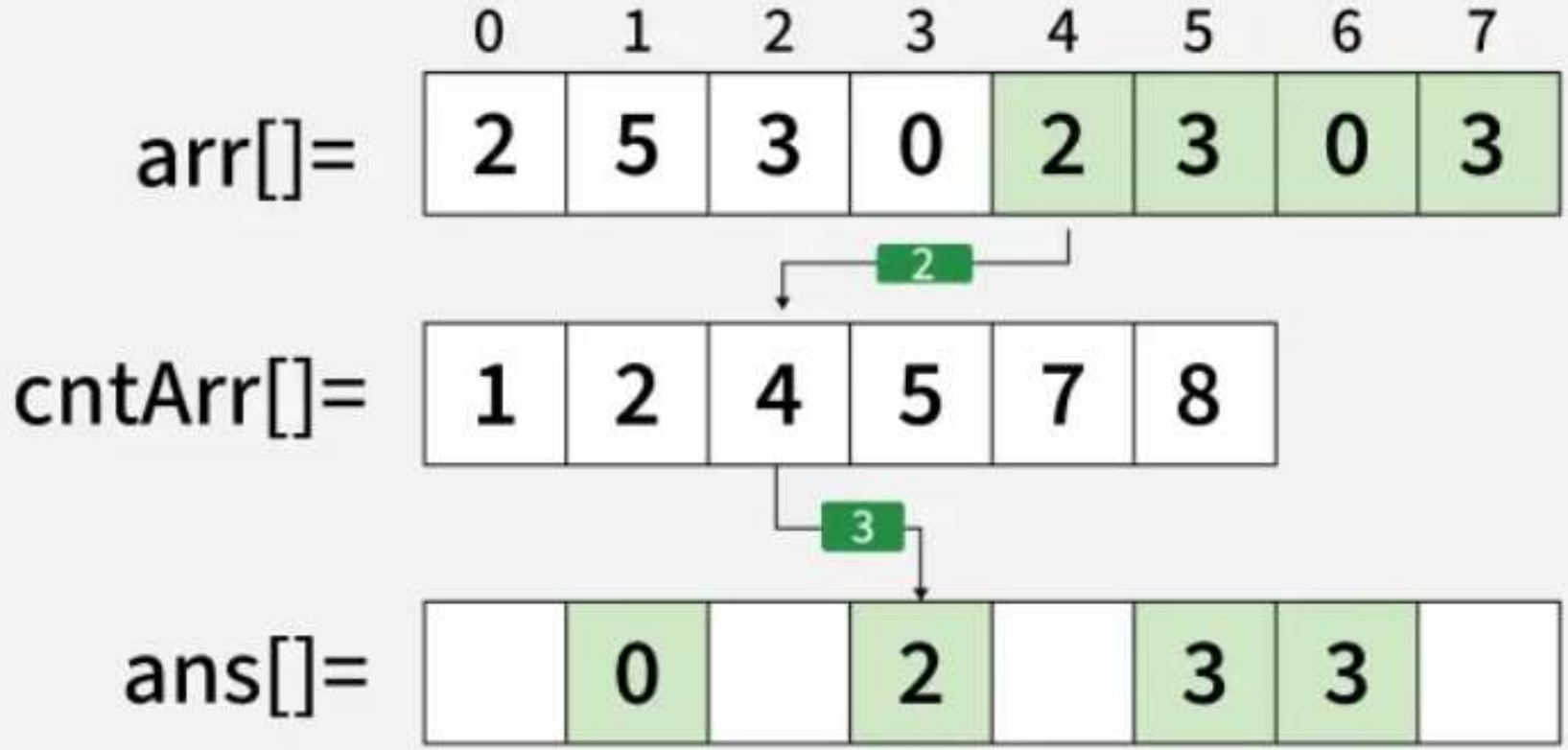
Update ans[cntArr[arr[6]] - 1] =arr[6]
Also, update cntArr[arr[6]] = cntArr[arr[6]]- -



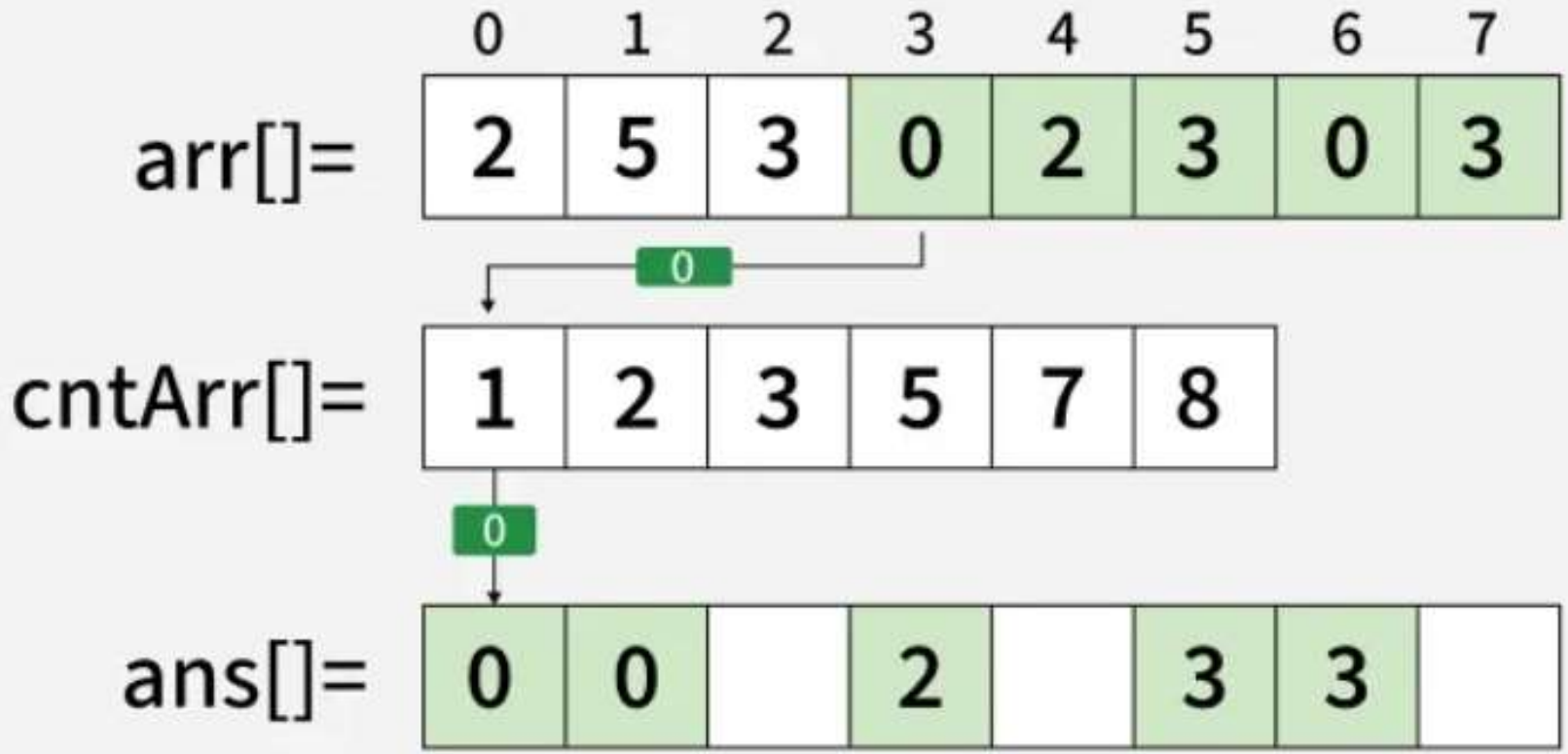
Update ans[cntArr[arr[5]] - 1] =arr[5]
Also, update cntArr[arr[5]] = cntArr[arr[5]]- -



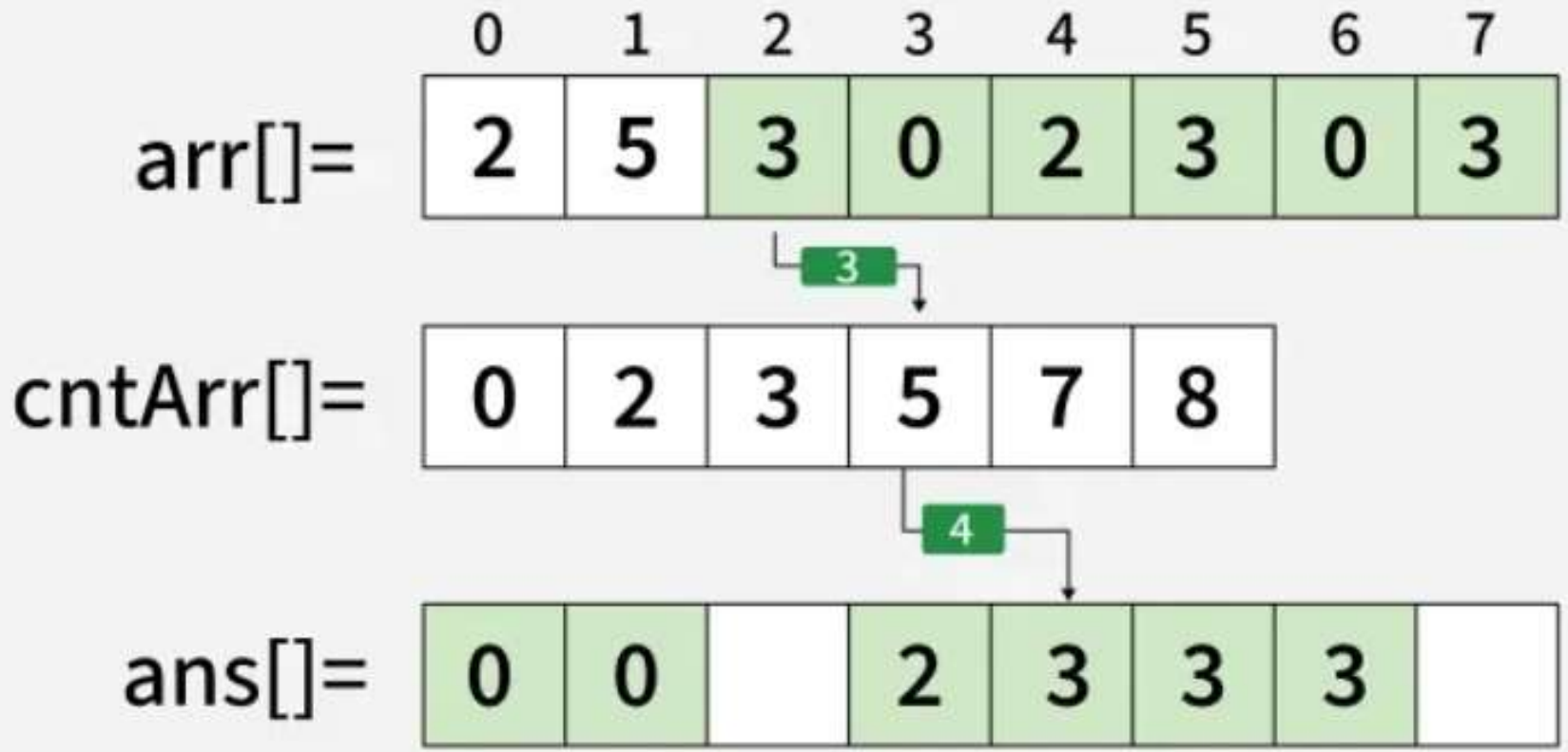
Update ans[cntArr[arr[4]] - 1] =arr[4]
Also, update cntArr[arr[4]] = cntArr[arr[4]]- -



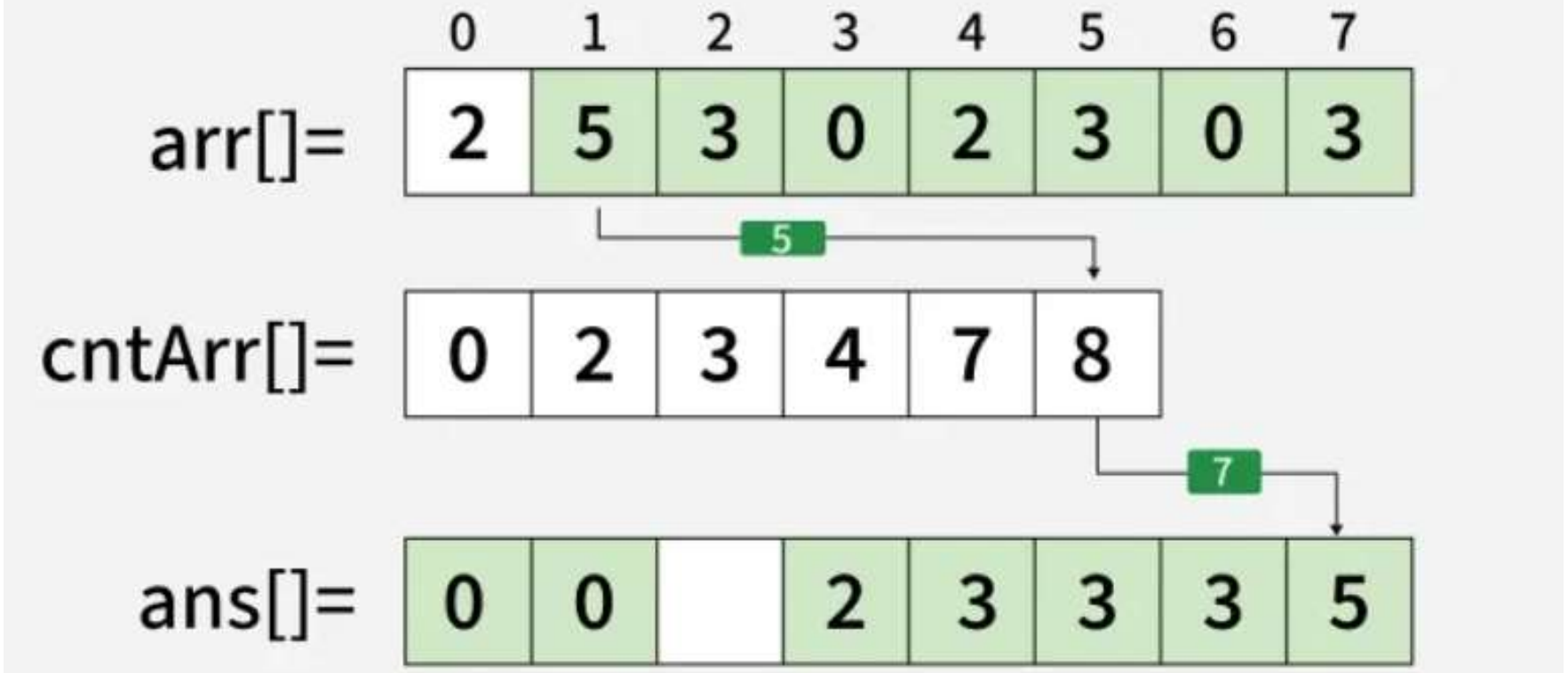
Update ans[cntArr[arr[3]] - 1] =arr[3]
Also, update cntArr[arr[3]] = cntArr[arr[3]]- -



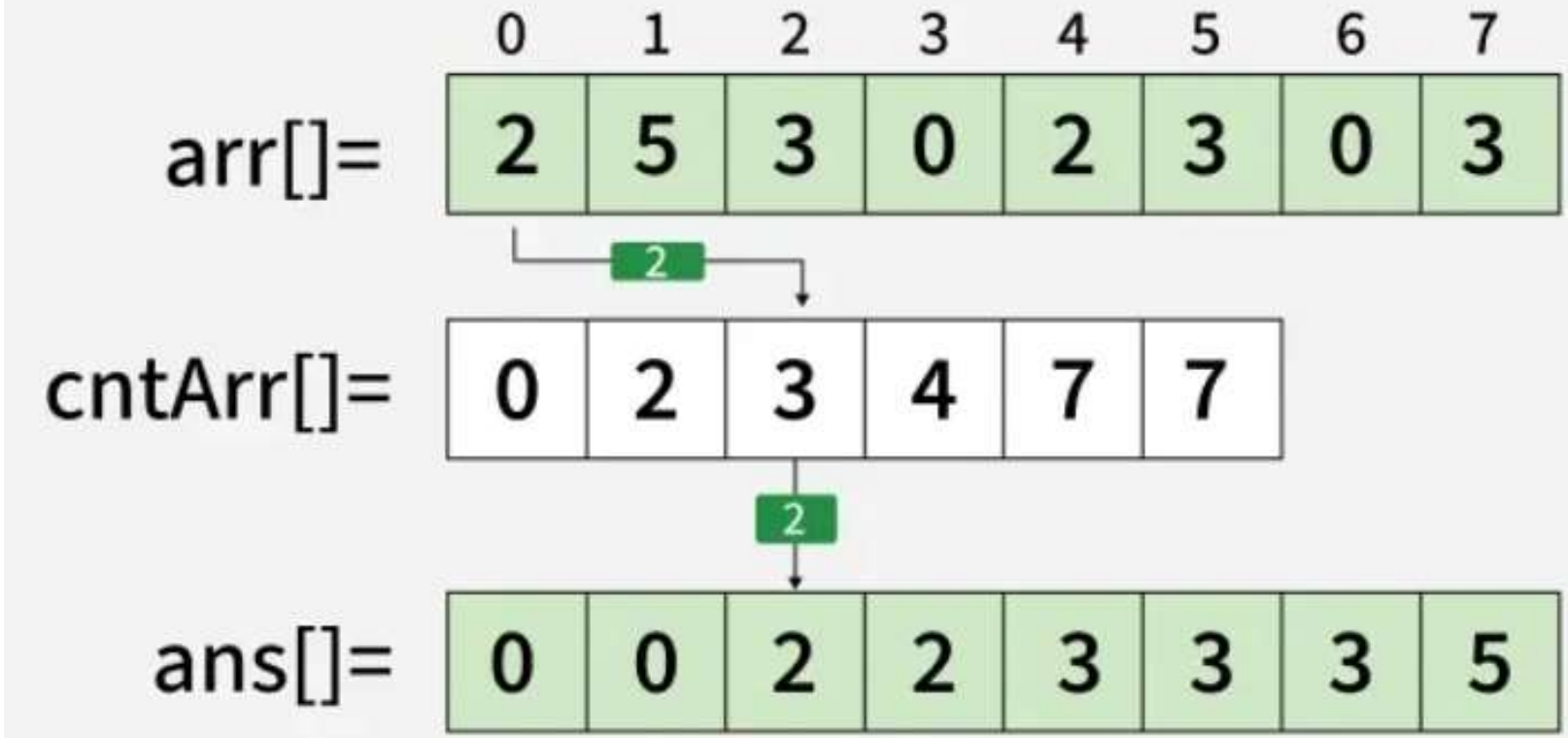
Update ans[cntArr[arr[2]] - 1] =arr[2]
Also, update cntArr[arr[2]] = cntArr[arr[2]]- -



Update ans[cntArr[arr[1]] - 1] =arr[1]
Also, update cntArr[arr[1]] = cntArr[arr[1]]- -



Update ans[cntArr[arr[0]] - 1] =arr[0]
Also, update cntArr[arr[0]] = cntArr[arr[0]]- -



```cpp
#include <iostream>
#include <vector>
using namespace std;

vector<int> countsort(vector<int>& arr) {
    int n = arr.size();

    // find the maximum element
    int maxval = 0;
    for (int i = 0; i < n; i++)
        maxval = max(maxval, arr[i]);

    // create and initialize cntArr array
    vector<int> cntArr(maxval + 1, 0);

    // count frequency of each element
    for (int i = 0; i < n; i++)
        cntArr[arr[i]]++;

    // compute prefix sum
    for (int i = 1; i <= maxval; i++)
        cntArr[i] += cntArr[i - 1];
```

```
    // build output array
    vector<int> ans(n);
    for (int i = n - 1; i >= 0; i--) {
        ans[cntArr[arr[i]] - 1] = arr[i];
        cntArr[arr[i]]--;
    }

    return ans;
}

int main() {
    vector<int> arr = {2,5,3,0,2,3,0,3};
    vector<int> ans = countsort(arr);

    for (int x : ans)
        cout << x << " ";

    return 0;
}
```

Bucket sort

- The Bucket Sort algorithm is **similar to the Counting Sort** algorithm, as it is just the generalized form of the counting sort.
- Bucket sort assumes that the input elements are drawn from a uniform distribution over the interval [0, 1].
- Hence, the bucket sort algorithm divides the interval [0, 1) into n equal parts, and the input elements are added to indexed *buckets* where the **indices based on the lower bound** of the (n element) value.
- Since the algorithm assumes the values as the independent numbers evenly distributed over a small range, not many elements fall into one bucket only.

Bucket Sort Algorithm

Let us look at how this algorithm would proceed further below –

Step 1 – Divide the interval in n equal parts, each part being referred to as a bucket. Say if n is 10, then there are 10 buckets; otherwise more.

Step 2 – Take the input elements from the input array A and add them to these output buckets B based on the computation formula, $B[i] = \lfloor n.A[i] \rfloor$

Step 3 – If there are any elements being added to the already occupied buckets, created a linked list through the corresponding bucket.

Step 4 – Then we apply insertion sort to sort all the elements in each bucket.

Step 5 – These buckets are concatenated together which in turn is obtained as the output.

Example

Consider, an input list of elements, 0.78, 0.17, 0.93, 0.39, 0.26, 0.72, 0.21, 0.12, 0.33, 0.28, to sort these elements using bucket sort −

Solution

Step 1

Linearly insert all the elements from the index 0 of the input array. That is, we insert 0.78 first followed by other elements sequentially. The position to insert the element is obtained using the formula − B[i]= ⌊ n.A[i]⌋, i.e, ⌊10 0.78⌋=7

0	
1	
2	
3	
4	
5	
6	
7	0.78
8	
9	

Now, we insert 0.17 at index $\lfloor 10\ 0.17 \rfloor = 1$

0	
1	0.17
2	
3	
4	
5	
6	
7	0.78
8	
9	

Step 3

Inserting the next element, 0.93 into the output buckets at $\lfloor 10\ 0.93 \rfloor = 9$

0	
1	0.17
2	
3	
4	
5	
6	
7	0.78
8	
9	0.93

Step 4

Insert 0.39 at index 3 using the formula $\lfloor 10\ 0.39 \rfloor = 3$

0	
1	0.17
2	
3	0.39
4	
5	
6	
7	0.78
8	
9	0.93

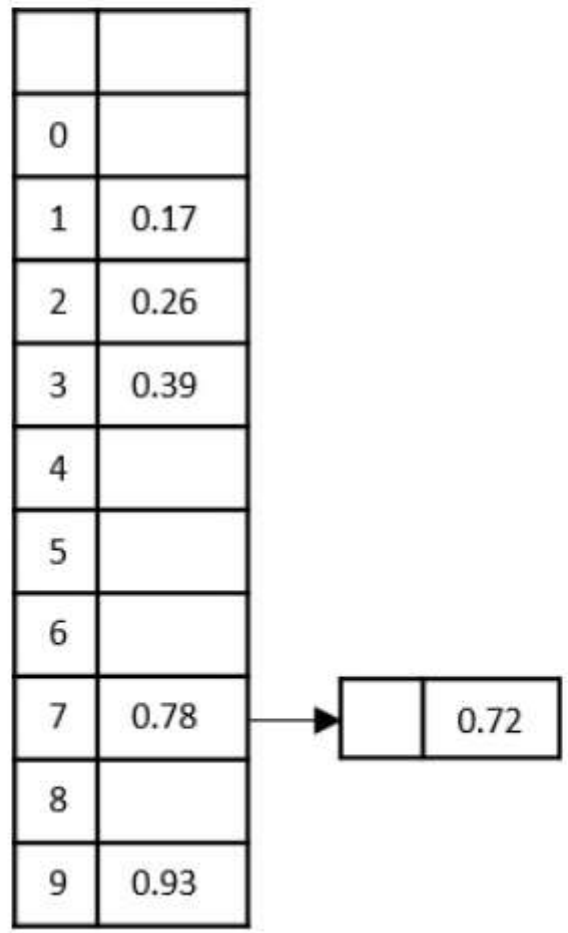
Step 5

Inserting the next element in the input array, 0.26, at position $\lfloor 10\ 0.26 \rfloor = 2$

0	
1	0.17
2	0.26
3	0.39
4	
5	
6	
7	0.78
8	
9	0.93

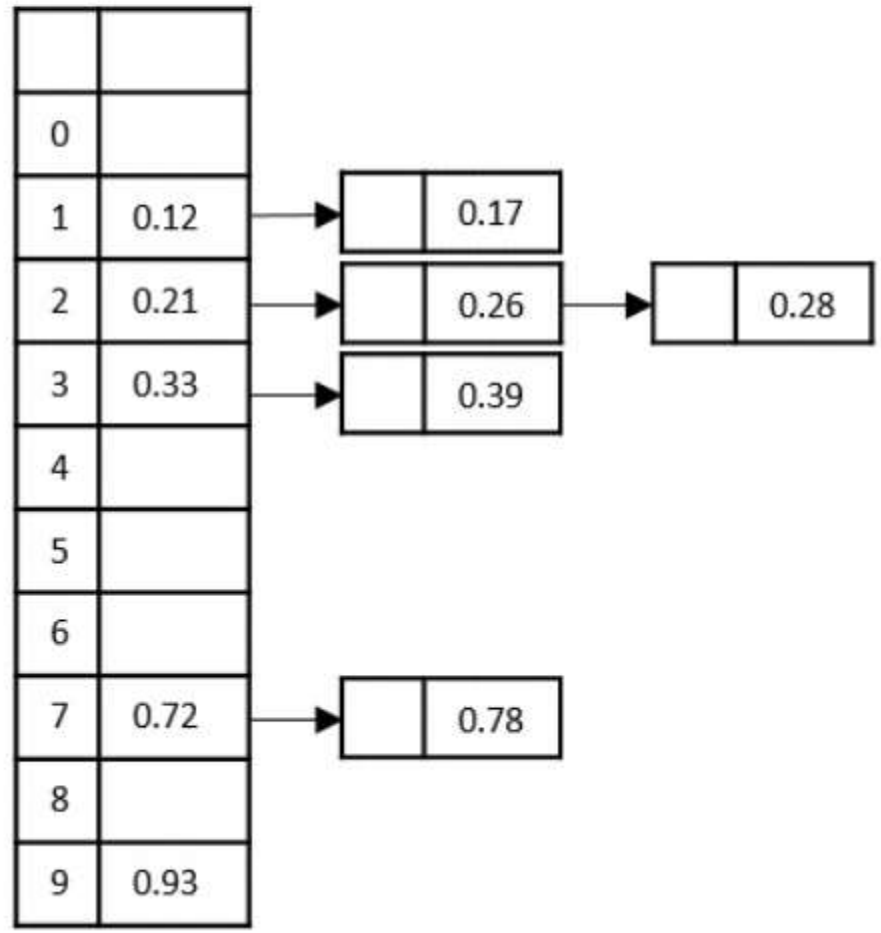
Step 6

Here is where it gets tricky. Now, the next element in the input list is 0.72 which needs to be inserted at index 7 using the formula $\lfloor 10\ 0.72 \rfloor = 7$. But theres already a number in the 7th bucket. So, a link is created from the 7th index to store the new number like a linked list, as shown below –



Step 7

Add the remaining numbers to the buckets in the similar manner by creating linked lists from the desired buckets. But while inserting these elements as lists, we apply insertion sort, i.e., compare the two elements and add the minimum value at the front as shown below −



Step 8

Now, to achieve the output, concatenate all the buckets together.

0.12, 0.17, 0.21, 0.26, 0.28, 0.33, 0.39, 0.72, 0.78, 0.93

```cpp
#include <iostream>
using namespace std;
void bucketsort(int a[], int n){ // function to implement bucket sort
   int max = a[0]; // get the maximum element in the array
   for (int i = 1; i < n; i++)
      if (a[i] > max)
         max = a[i];
   int b[max], i;
   for (int i = 0; i <= max; i++) {
      b[i] = 0;
   }
   for (int i = 0; i < n; i++) {
      b[a[i]]++;
   }
   for (int i = 0, j = 0; i <= max; i++) {
      while (b[i] > 0) {
         a[j++] = i;
         b[i]--;
      }
```

```
int main(){
   int a[] = {12, 45, 33, 87, 56, 9, 11, 7, 67};
   int n = sizeof(a) / sizeof(a[0]); // n is the size of array
   cout << "Before sorting array elements are: \n";
   for (int i = 0; i < n; ++i)
      cout << a[i] << " ";
   bucketsort(a, n);
   cout << "\nAfter sorting array elements are: \n";
   for (int i = 0; i < n; ++i)
      cout << a[i] << " ";
}
```

Radix Sort

- Radix sort is a step-wise sorting algorithm that **starts the sorting from the least significant digit of the input elements. (unit place in the digit)**
- These least significant digits **(Unit place) are all considered individual elements** and sorted first;
- followed by the second least significant digits (10’s place in the digit). This process is continued until all the digits of the input elements are sorted.

Note − If the elements do not have same number of digits, find the maximum number of digits in an input element and add leading zeroes to the elements having less digits. It does not change the values of the elements but still makes them k-digit numbers.

Radix Sort Algorithm

The radix sort algorithm makes use of the counting sort algorithm while sorting in every phase. The detailed steps are as follows −

Step 1 − Check whether all the input elements have same number of digits. If not, check for numbers that have maximum number of digits in the list and add leading zeroes to the ones that do not.

Step 2 − Take the least significant digit of each element.

Step 3 − Sort these digits using counting sort logic and change the order of elements based on the output achieved. For example, if the input elements are decimal numbers, the possible values each digit can take would be 0-9, so index the digits based on these values.

Step 4 − Repeat the Step 2 for the next least significant digits until all the digits in the elements are sorted.

Step 5 − The final list of elements achieved after kth loop is the sorted output.

Example

For the given unsorted list of elements, 236, 143, 26, 42, 1, 99, 765, 482, 3, 56, we need to perform the radix sort and obtain the sorted output list −

Step 1

Check for elements with maximum number of digits, which is 3. So we add leading zeroes to the numbers that do not have 3 digits. The list we achieved would be −

```
236, 143, 026, 042, 001, 099, 765, 482, 003, 056
```

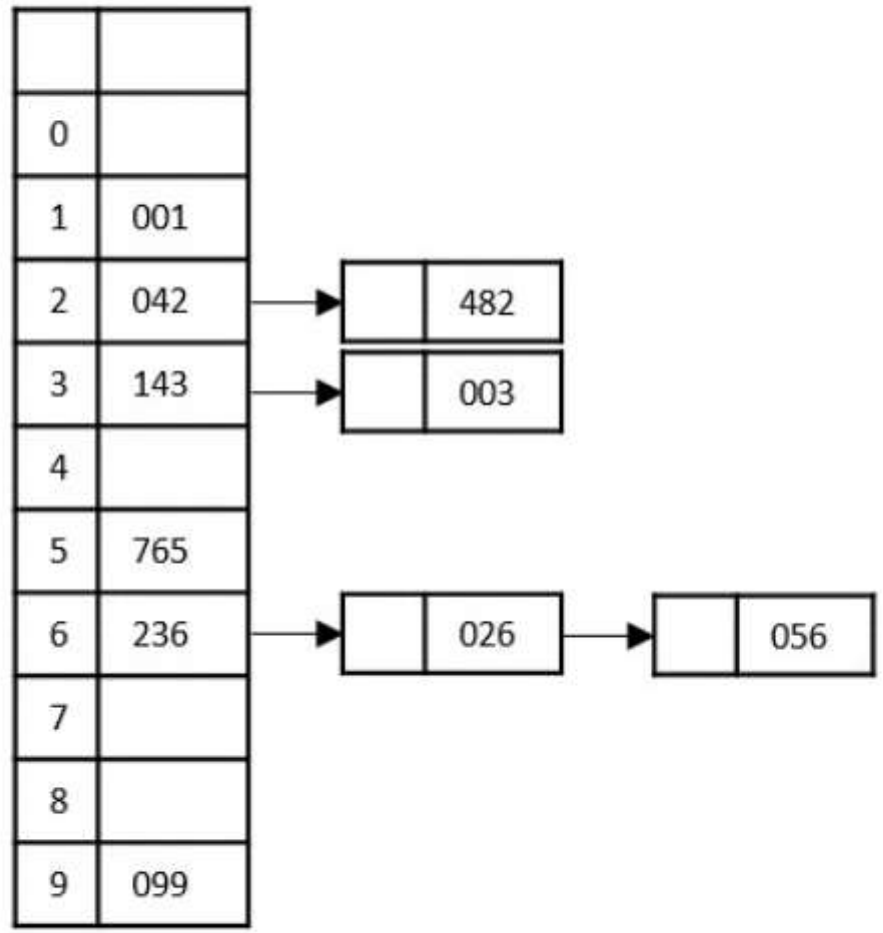
Step 2

Construct a table to store the values based on their indexing. Since the inputs given are decimal numbers, the indexing is done based on the possible values of these digits, i.e., 0-9.

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

Step 3

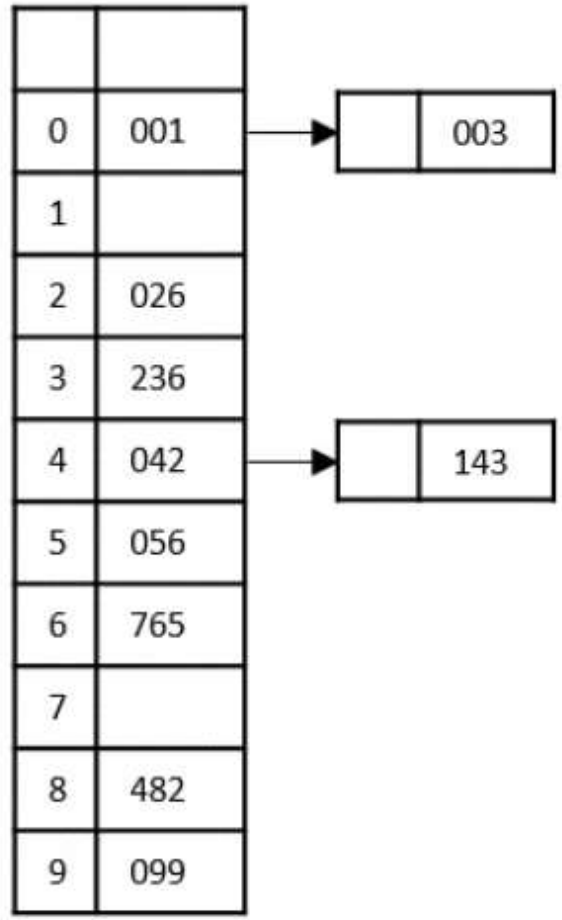
Based on the least significant digit of all the numbers, place the numbers on their respective indices.



The elements sorted after this step would be 001, 042, 482, 143, 003, 765, 236, 026, 056, 099.

Step 4

The order of input for this step would be the order of the output in the previous step. Now, we perform sorting using the second least significant digit.



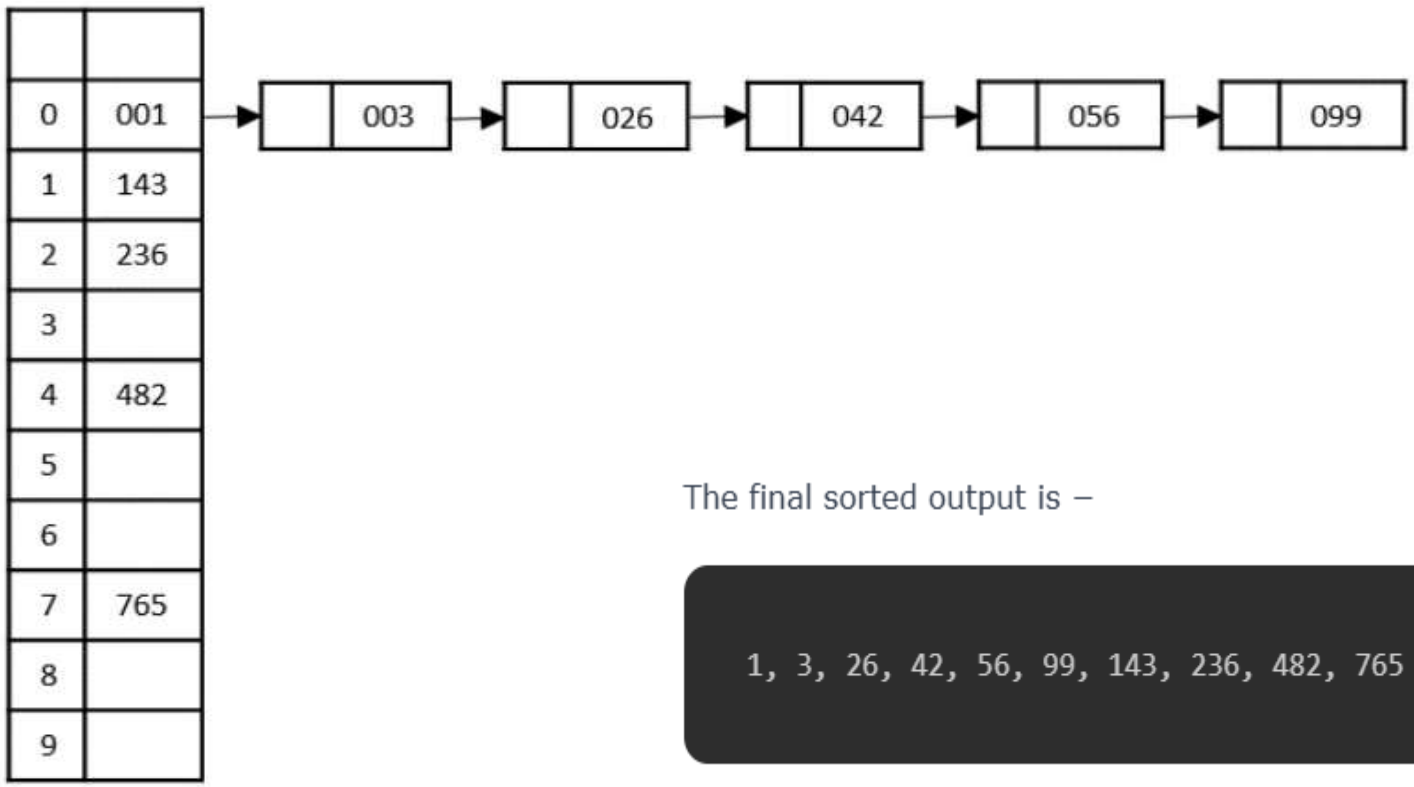
The order of the output achieved is 001, 003, 026, 236, 042, 143, 056, 765, 482, 099.

Step 5

The input list after the previous step is rearranged as −

```
001, 003, 026, 236, 042, 143, 056, 765, 482, 099
```

Now, we need to sort the last digits of the input elements.



The final sorted output is −

```
1, 3, 26, 42, 56, 99, 143, 236, 482, 765
```

Since there are no further digits in the input elements, the output achieved in this step is considered as the final output.

```
using namespace std;
void countsort(int a[], int n, int pos){
    int output[n + 1];
    int max = (a[0] / pos) % 10;
    for (int i = 1; i < n; i++) {
        if (((a[i] / pos) % 10) > max)
            max = a[i];
    }
    int count[max + 1];
    for (int i = 0; i < max; ++i)
        count[i] = 0;
    for (int i = 0; i < n; i++)
        count[(a[i] / pos) % 10]++;
    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];
    for (int i = n - 1; i >= 0; i--) {
        output[count[(a[i] / pos) % 10] - 1] = a[i];
        count[(a[i] / pos) % 10]--;
    }
```

```
void radixsort(int a[], int n){
    int max = a[0];
    for (int i = 1; i < n; i++)
        if (a[i] > max)
            max = a[i];
    for (int pos = 1; max / pos > 0; pos *= 10)
        countsort(a, n, pos);
}
int main(){
    int a[] = {236, 15, 333, 27, 9, 108, 76, 498};
    int n = sizeof(a) / sizeof(a[0]);
    cout <<"Before sorting array elements are: ";
    for (int i = 0; i < n; ++i) {
        cout <<a[i] << " ";
    }
    radixsort(a, n);
    cout <<"\nAfter sorting array elements are: ";
    for (int i = 0; i < n; ++i) {
        cout << a[i] << " ";
```